

UID ASSIGNMENT 02

COMPUTER NETWORKS

Name	Kushagra Vashistha
UID	24Bai70915
Section	24AIT-KRG-1B
Subject	Computer Networks
Assignment	02
Submission Date	27 September 2026

Q1. What is an IP Address? Why do we need classes in case of IPv4 address? Explain each class, mention how many hosts and networks are possible in each class. Also explain subnetting and supernetting with examples.

What is an IP Address?

An **IP (Internet Protocol) address** is a unique logical address assigned to a device connected to a network. It is used to identify the device and facilitate communication between devices over a network.

IPv4 uses a **32-bit address**, generally represented as four decimal numbers separated by dots.

Example: 192.168.1.10

Each part is called an **octet** and can have a value from 0 to 255.

A 32-bit IPv4 address can be divided into **Network ID** and **Host ID**. The Network ID identifies the network, while the Host ID identifies a particular device within that network.

Why were classes needed in IPv4?

IPv4 originally used a **classful addressing system** to divide the available 32-bit address space into different categories. The main reason was to accommodate networks of different sizes. A very large organization might require thousands of host addresses, while a small organization might require only a few dozen.

Therefore, IPv4 was divided into **Class A, B, C, D, and E**. Classes A, B, and C were used for traditional unicast addressing, Class D for multicast, and Class E was reserved for experimental purposes.

IPv4 Address Classes

Class	First Octet	Default Mask	Network Bits	Host Bits	Networks*	Hosts/Network*
A	1–126	255.0.0.0 (/8)	8	24	126	16,777,214
B	128–191	255.255.0.0 (/16)	16	16	16,384	65,534
C	192–223	255.255.255.0 (/24)	24	8	2,097,152	254
D	224–239	—	—	—	Multicast	—
E	240–255	—	—	—	Experimental	—

*For traditional classful IPv4 calculations. Class A excludes 0 and 127 from normal classful network allocation. Host counts exclude the all-zero network address and all-one broadcast address.

Class A

Class A was designed for very large networks. Its default subnet mask is 255.0.0.0. Example: 10.20.30.40. There are 126 usable Class A networks and 16,777,214 usable host addresses per network under traditional classful addressing.

Class B

Class B was designed for medium-sized networks. Its default subnet mask is 255.255.0.0. Example: 172.16.20.10. There are $2^{14} = 16,384$ networks and $2^{16} - 2 = 65,534$ hosts per network.

Class C

Class C was designed for small networks. Its default subnet mask is 255.255.255.0. Example: 192.168.1.25. There are $2^{21} = 2,097,152$ networks and $2^8 - 2 = 254$ hosts per network.

Class D

Class D addresses range from 224.0.0.0 to 239.255.255.255 and are used for multicast communication. They are not divided into network and host portions in the traditional classful sense.

Class E

Class E addresses range from 240.0.0.0 to 255.255.255.255. They were reserved for experimental and research purposes and are not used for ordinary host addressing.

Subnetting

Subnetting is the process of dividing one large network into multiple smaller logical networks called subnets. It is done by borrowing bits from the host portion and using them as subnet bits.

Example: Suppose we have 192.168.1.0/24. If we divide it into four equal subnets, we borrow 2 host bits, changing /24 to /26. Each subnet contains 64 total addresses.

Subnet	Network Address	Usable Hosts	Broadcast
1	192.168.1.0/26	192.168.1.1–62	192.168.1.63
2	192.168.1.64/26	192.168.1.65–126	192.168.1.127
3	192.168.1.128/26	192.168.1.129–190	192.168.1.191
4	192.168.1.192/26	192.168.1.193–254	192.168.1.255

Subnetting improves address management, network organization, and traffic isolation.

Supernetting

Supernetting is the process of combining multiple smaller, contiguous networks into a larger logical network. It is also called **route aggregation** or **route summarization**.

Example: The four networks 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24, and 192.168.3.0/24 can be combined into 192.168.0.0/22. Instead of maintaining four separate routing entries, a router can advertise one summarized route.

Difference: Subnetting converts one large network into multiple smaller networks, while supernetting combines multiple smaller networks into one larger logical network.

Q2. What is the difference between Link State and Distance Vector routing? Explain how these two algorithms help in identifying the least-cost path with the help of examples.

Routing

Routing is the process of determining the path through which packets should travel from a source network to a destination network. A routing algorithm attempts to find an efficient or **least-cost path**. Cost may represent the number of hops, delay, bandwidth, reliability, or administrative cost.

Distance Vector Routing

In Distance Vector routing, each router maintains information about the distance or cost to destinations and the next router to use. Routers exchange routing information with their neighboring routers. A common algorithm associated with this approach is the **Bellman-Ford algorithm**.

Example: Consider A–B with cost 2, B–D with cost 1, A–C with cost 5, and C–D with cost 2. The path A → B → D has cost $2 + 1 = 3$, while A → C → D has cost $5 + 2 = 7$. Therefore, A selects A → B → D as the least-cost path.

Distance Vector routers gradually learn this information from their neighbors. The method is relatively simple, but it can converge slowly and may suffer from routing loops and the count-to-infinity problem.

Link State Routing

In Link State routing, every router attempts to build a map of the network topology. Routers discover their neighbors, determine the cost of each link, advertise this information, build a topology database, and run a shortest-path algorithm.

The commonly associated algorithm is **Dijkstra's Shortest Path First (SPF)** algorithm.

Using the same example, A → B → D has cost 3 and A → C → D has cost 7. Therefore, the least-cost path is A → B → D with total cost 3.

Link State vs Distance Vector

Feature	Distance Vector	Link State
Basic idea	Shares distance information	Shares link-state information
Knowledge	Mainly information from neighbors	Builds broader network topology
Algorithm	Bellman-Ford	Dijkstra
Updates	Periodic/triggered depending on protocol	Link-state advertisements/flooding
Convergence	Generally slower	Generally faster
Network view	Limited	Detailed topology
Complexity	Lower	Higher
Example protocol	RIP	OSPF

Distance Vector identifies paths by exchanging distance information with neighboring routers, while Link State identifies paths by constructing a topology database and applying a shortest-path algorithm.

Q3. What is congestion in a network? Why do we need to control congestion? Explain the difference between Leaky Bucket and Token Bucket with examples. Also, explain why we need to do fragmentation.

Network Congestion

Congestion occurs when the amount of traffic entering a network or router becomes greater than the network's ability to process and forward it.

In simple terms: Incoming Traffic > Network Capacity

When congestion occurs, routers may experience large queues, increased delay, packet loss, retransmissions, and reduced throughput.

Why do we need congestion control?

- Prevent excessive packet loss.
- Reduce network delay.
- Maintain good throughput.
- Prevent buffer overflow.
- Ensure fair use of network resources.
- Avoid congestion collapse.

Leaky Bucket Algorithm

The Leaky Bucket algorithm is a traffic-shaping mechanism that controls the rate at which packets are transmitted. It can be compared to a bucket with a small hole at the bottom. Packets can arrive at variable rates, but they leave at approximately a constant rate.

Example: Suppose bucket capacity = 5 packets and output rate = 1 packet/second. If 5 packets arrive suddenly, they are stored and transmitted one per second. If more packets arrive when the bucket is full, excess packets may be discarded.

Main idea: Variable input → Fixed-rate output.

Token Bucket Algorithm

In the Token Bucket algorithm, tokens are generated at a fixed rate and stored in a bucket. A packet can be transmitted when sufficient tokens are available.

Example: Suppose token generation = 2 tokens/second and bucket capacity = 10 tokens. If the bucket has accumulated tokens, a sudden burst of packets can be transmitted using those stored tokens. After the tokens are consumed, transmission follows the token generation rate.

Main idea: Controlled average rate with support for bursts.

Leaky Bucket vs Token Bucket

Feature	Leaky Bucket	Token Bucket
Output	Usually fixed rate	Variable rate within limits
Bursts	Generally does not allow large bursts	Allows controlled bursts
Mechanism	Packets wait in a bucket/queue	Tokens accumulate
Traffic shaping	Strong smoothing	Flexible traffic shaping
Main purpose	Produce a steady output rate	Control average rate while allowing bursts

What is Fragmentation?

Fragmentation is the process of dividing a large IP packet into smaller fragments so that it can pass through a network whose **Maximum Transmission Unit (MTU)** is smaller than the packet size.

Example: Suppose an IP packet is 4000 bytes and the next network has an MTU of 1500 bytes. The 4000-byte packet cannot be transmitted as a single packet over that link, so it may need to be divided into smaller fragments.

At the destination, the fragments can be reassembled into the original packet using information carried in the IP headers.

Why is fragmentation needed?

- Different networks can have different MTUs.
- A packet larger than the outgoing link's MTU cannot be transmitted as one unit.
- Fragmentation allows packets to travel across networks with smaller MTUs.
- It enables interoperability between networks having different maximum packet sizes.

Conclusion

IPv4 addressing provides a mechanism for identifying devices and networks. Classful addressing historically divided IPv4 addresses into different classes according to network size, while subnetting and supernetting provide ways to organize and aggregate address spaces.

Routing algorithms such as Distance Vector and Link State help routers determine efficient least-cost paths. Distance Vector relies on information exchanged between neighboring routers, while Link State builds a network topology and uses a shortest-path algorithm.

Network congestion occurs when traffic exceeds available network capacity. Leaky Bucket and Token Bucket algorithms help regulate traffic, while fragmentation allows IP packets to traverse networks having smaller MTUs.